Oblig 3: The `AeroScript` Language

Deadline: 30 October 2025

1 The Objective

For your third assignment, you will extend `AeroScript` with an event-mechanism that makes `AeroScript` more expressive. Regular execution in a mode can be interrupted by *events* that can trigger global modes. The assignment consists of adapting both the grammar, the abstract syntax tree and program execution accordingly.

2 Syntax

For easy reference, you can find the full concrete syntax of `AeroScript` in Figure 1. The main difference to the concrete syntax in Assignment 2 is that the production for *statement* has changed, and can now be either an *action* (as before), a *reaction*, or a *mode*. The new part of the concrete syntax is highlighted.

$$
\begin{array}{rcl}
 & n \in \mathbb{R}^{+} & \\
 & id \in \text{NAME} & \\
program & := & (mode)^{+} \\
mode & := & \text{`}\rightarrow\text{`}^{?}\ id\ \text{`\{`}\ (statement)^{*}\ \text{`\}`}\ (\text{`}\rightarrow\text{`}\ id)^{?} \\
statement & := & action \mid reaction \\
reaction & := & \text{`on`}\ event\ \text{`}\rightarrow\text{`}\ id \\
event & := & \text{`obstacle`} \mid \text{`low battery`} \mid \text{`message`}\ \text{`[`}\ id\ \text{`]`} \\
action & := & (acDock \mid acMove \mid acTurn \mid acAscend \mid acDescend) \\
 & & (\text{`for`}\ expression\ \text{`seconds`} \mid \text{`at speed`}\ expression)^{?} \\
acDock & := & \text{`return to base`} \\
acMove & := & \text{`move`}\ (\text{`to`}\ \text{`point`}\ point \mid \text{`by`}\ n) \\
acTurn & := & \text{`turn`}\ (\text{`right`} \mid \text{`left`})^{?}\ \text{`by`}\ expression \\
acAscend & := & \text{`ascend by`}\ expression \\
acDescend & := & \text{`descend by`}\ expression \mid \text{`descend to ground`} \\
expression & := & \text{`}-\text{`}expression \mid expression \oplus expression \mid \text{`random`}\ range^{?} \\
 & & \mid \text{`point`}\ point \mid n \mid \text{`(`}expression\text{`)`} \\
range & := & \text{`[`}expression\text{`,`}expression\text{`]`} \\
point & := & \text{`(`}expression\text{`,`}expression\text{`)`} \\
\oplus & := & \text{`+`} \mid \text{`-`} \mid \text{`*`}
\end{array}
$$

Figure 1. The syntax of `AeroScript`. The star ($\odot^{*}$) denotes a sequence containing 0 or more occurrences. and the question mark ($\odot^{?}$) denotes choice (zero or one occurrence), the plus ($\odot^{+}$) denotes a sequence containing one or more occurrences. NAME is the set of all possible identifiers.

Intuition: An `AeroScript` program consists of one or more *modes*, which have some similarity to methods in Java. Each mode is defined by optionally starting with a transition arrow, followed by an identifier and enclosed actions, reactions, or nested modes within braces. A mode may be prefixed by an optional arrow and may end with an optional arrow and a named mode. If a mode is prefixed by an arrow, this mode should be automatically started as part of the program's run block or as part of the current mode's actions.If a mode ends with an arrow to another mode, program execution should

transition to this mode and execute the corresponding instructions, before continuing with the rest of the current actions. The new event-mechanism allows a program to transition between different modes based on specified events. When program execution is evented, the remiaing actions of the current mode should be discarded. The following an example modes:

```
-> MissionControl {
  on message [launch] -> TakeOffLand
}

TakeOffLand {
  ascend by 20
  on low battery -> EmergencyLanding
  move to point (50, 50)
} -> ScanArea

ScanArea {
  move to point (100, 100) at speed 10
  on low battery -> EmergencyLanding
} -> EndMission

EndMission { return to base }

EmergencyLanding { descend to ground }
```

And another example:

```
-> RandomTour {
  on message [start_tour] -> TakeOff
}

TakeOff {
  ascend by 20
  on low battery -> EmergencyLanding
  move to point (random[0, 100], random[0, 100])
} -> Explore

Explore {
  move to point (random[0, 100], random[0, 100]) at speed 10
  on low battery -> EmergencyLanding
  on message [end_tour] -> ReturnToBase
}

ReturnToBase { return to base }

EmergencyLanding { descend to ground }
```

When executing an `AeroScript` program, your program should read the content of the file `program.aero` (can be found under `src/test/resources/`) and parse it into an AST. Your program should then execute the `AeroScript` program by traversing the AST and executing the statements.

Note: reactions (message and battery) **should not** be executed unless they are triggered by an event. Moreover, the modes (e.g., `TakeOffLand`, `TakeOff`, etc.) **must not** be executed unless they are invoked (e.g., `TakeOffLand` gets executed when we receive the message `launch`).

After all the statements are executed, your program needs to enter the `REPL` and wait for triggers (messages). When a trigger is received, your program should execute the corresponding reaction. The program can either execute all the statements that are not "free" to execute, or can block at the first message and then wait for an incoming message to continue the iteration. To end the simulation, you must type `exit` in the REPL. Until then, the program must execute all the statements that have to be executed and then wait.

3 Semantics of `AeroScript`

The syntax and semantics of `AeroScript` expressions, actions and modes were covered (and hopefully implemented) in your previous assignments; we now focus on how to evaluate reactions and modes.

Every mode is a sequence of actions and reactions. When a mode is entered, the actions are executed in order. If a reaction is triggered, remaining actions of the current mode are discarded and the program transitions to the mode specified in the reaction. If no reaction is triggered, the program transitions to the next statement in the sequence. At all times the program needs to keep track at the current

- position,
- altitude,
- distance traveled, and
- battery level.

If the level of the battery goes below 20, the program should trigger the `low battery` event if present, or exit with a failure otherwise.

Each action needs to be printed on the command line as a log, and will affect the variables of the program (be it the position as coordinate in X and Y or the altitude if ascending or descending) and will deplete the battery level.

3.1 Detailed semantics

Modes and Transitions In `AeroScript`, **modes** are like different states or scenarios in which a drone can operate—such as taking off, flying, or landing and are defined in the grammar by the `mode`. Each mode contains a set of operations that can be executed when the drone is in that state. For instance, a `TakeOff` mode might include instructions to power up the rotors and ascend to a specific altitude. Note that in order to have a program that makes the drone *do* something, it must have at least one outer mode with a leading transition arrow—the *start mode*.

- **Transitions** between modes are triggered by events or conditions. For example, once a drone reaches the desired altitude in `TakeOff` mode, it might automatically switch to a `Hover` or `Navigate` mode.

Events and Reactions Events are specific occurrences like detecting an obstacle, receiving a command to return to base, or having low battery. In `AeroScript`, you define what the drone should do when these events happen through **reactions**.

- If an event like `on low battery` is triggered, the drone might switch to a `ReturnToBase` mode, which contains a predefined set of actions to safely return and land the drone.

Actions, Expressions and Calculations Actions, expressions and calculations were covered in the previous assignments and work as before; in particular, the computation of current position, battery depletion and distance travelled works as before. No changes are warranted here!

3.2 Putting It All Together

After parsing the source into an AST, your interpreter should traverse it and execute the instructions. More specifically, it should identify the start mode, execute actions, wait for events, and evaluate expressions. The interpreter continually checks which mode the drone is in, what events might trigger a mode change, and which actions to execute based on the current mode and events.

After all the statements are executed, the program needs to enter the REPL and wait for triggers (messages). When a trigger is received, the program should execute the corresponding reaction. The program should continue to wait for triggers until the user types `exit` in the REPL.

Note: The reactions (message and battery) **do not** have to be executed if they are not triggered. Moreover, the functions that are defined (e.g. TakeOffLand, TakeOff, etc.) **must not** be executed when not invoked (e.g. `TakeOffLand` gets executed when we receive the message `launch`). Modes that start with '$->$' will have to be executed when reached. Modes that do not start with '$->$' should not be executed unless they are invoked by a message, or the are invoked by a previous mode (those are marked with the '$\}-> ID$', with `ID` being the name of the following mode that will have to be invoked).

4 Your task

For this assignment you must extend your work from the last assignment or use the given expression semantics. In particularm you are expected to do the following:

- Extend the AST to cover the entire syntax of `AeroScript`.
- Extend the interpreter accordingly.
- Extend the lexer and parser to cover the whole language using `ANTLR`.
- We have provided some `JUnit` tests. **Discuss the coverage of the tests**, and provide your own.
- Write a report (3-4 pages) where you discuss the challenges you faced and choices you made during implementation. The report should cover the entire work (that is also the most important part of assignments 1 and 2), but should have a focus on this work. Be sure to read the report guide on the course page beforehand.

5 Implementation Hints

- If you want to use listeners, you can see the REPL class for an example of how to implement them and use the code commented. However, you can also use a different approach if you prefer.
- After you `exit` the REPL, the program should print the initial position, initial battery capacity, final position, final battery level and distance travelled
- When implementing the events, make sure to add the correct logic for each event:
  - `obstacle` should trigger the function corresponding to the event immediately
  - `low battery` should trigger the function corresponding to the event if the battery level is below 20
  - `message` should trigger the function corresponding to the event if the message is received from the REPL
- When writing the grammar remember to always have the correct mapping for the keywords (e.g. ON for 'on', BATTERY for 'low battery', etc.) so that you can then check for their existence in the `Interpreter`

6 What to Submit

You should hand in a `zip` file containing your report (as a PDF, about 3-4 pages) along with the project containing your implementation. Make sure to clean (run `./gradlew clean`) any unnecessary garbarge files before zipping. If your zip-file is very large, you likely did not clean.

7 Where to Submit

In IN2031, we use Devilry, not Canvas, for assignments. Go to

https://devilry.ifi.uio.no

and log in with your user UiO-name and password. When you are logged in, it should be fairly obvious how to upload the report and code.